

Level 1 Challenge 91

▼ Mandatory Test Cases

Test Case 1

KEYWORD

```
for(i=0;i<n;i++)
```

Test Case 2

KEYWORD

```
if(arr[i]>max)
```

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

9

Test Case 2

TOKEN COUNT

330

Test Case 3

NLOC

55

Hashing1.c +

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define MAXN 200005
5
6 typedef struct {
7     int val;
8     int idx;
9 } Element;
10
11 int cmp(const void *a, const void *b) {
12     Element *e1 = (Element*)a;
13     Element *e2 = (Element*)b;
14     if (e1->val != e2->val)
15         return e1->val - e2->val;
16     return e1->idx - e2->idx;
17 }
18
19 int main() {
20     int n;
21     if (scanf("%d", &n) != 1) return 0;
22
23     int *arr = (int*)malloc(n * sizeof(int));
24     int max = 0;
25     int i;
```

```
26     for(i=0;i<n;i++) {
27         scanf("%d", &arr[i]);
28         if(arr[i]>max) {
29             max = arr[i];
30         }
31     }
32
33     Element *elems = (Element*)malloc(n * sizeof(Element));
34     for (i = 0; i < n; i++) {
35         elems[i].val = arr[i];
36         elems[i].idx = i;
37     }
38     qsort(elems, n, sizeof(Element), cmp);
39
40     int *compressed = (int*)malloc(n * sizeof(int));
41     int distinct_cnt = 0;
```

```

42     for (i = 0; i < n; i++) {
43         if (i == 0 || elems[i].val != elems[i - 1].val) {
44             distinct_cnt++;
45         }
46         compressed[i].idx = distinct_cnt - 1;
47     }
48
49     int *first_pos = (int*)malloc(distinct_cnt * sizeof(int));
50     for (i = 0; i < distinct_cnt; i++) first_pos[i] = -1;
51     for (i = 0; i < n; i++) {
52         int c = compressed[i];
53         if (first_pos[c] == -1) {
54             first_pos[c] = i;
55         }
56     }
57

```

main | CH.SC.U4CSE25225 | Hashing1.c

Ln 89, Col 1 Spaces: 4 UTF-8 CRLF C

```

58  int *seen = (int*)calloc(distinct_cnt, sizeof(int));
59  int *suffix_distinct = (int*)malloc((n + 1) * sizeof(int));
60  suffix_distinct[n] = 0;
61  for (i = n - 1; i >= 0; i--) {
62      int c = compressed[i];
63      if (!seen[c]) {
64          seen[c] = 1;
65          suffix_distinct[i] = suffix_distinct[i + 1] + 1;
66      } else {
67          suffix_distinct[i] = suffix_distinct[i + 1];
68      }
69  }
70
71  long long total_unique_pairs = 0;
72  for (int u = 0; u < distinct_cnt; u++) {
73      int pos = first_pos[u];

```

```
74     if (pos + 1 < n) {
75         total_unique_pairs += suffix_distinct[pos + 1];
76     }
77 }
78
79 printf("%lld\n", total_unique_pairs);
80
81 free(arr);
82 free(elems);
83 free(compressed);
84 free(first_pos);
85 free(seen);
86 free(suffix_distinct);
87 return 0;
88}
89
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

5

1 2 2 1 3

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Hashing1.exe

6

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

7

1 4 1 2 2 1 3

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Hashing1.exe

10

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>